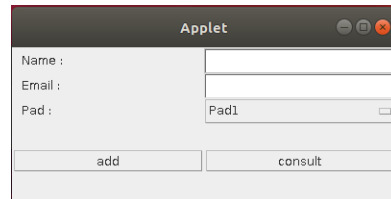


RMI lab

Daniel Hagimont (daniel.hagimont@irit.fr)

The objective of this lab is to practice with Java RMI to implement a simple application where a graphical applet allows to enter the coordinates of a person (name, email) and to register them in a remote server. The GUI of the applet allows selecting the server in which the user wants to register the person. We will manage only 2 servers.

The application also allows to lookup for a person given his name, indicating the server where this lookup should be performed.



If the person is not found in the indicated server, the search should be propagated to the other server. The interface of the server class is the following:

```
public interface Pad extends Remote {
    public void add(SRecord sr) throws RemoteException;
    public RRecord consult(String n, boolean forward) throws RemoteException;
}
public interface SRecord extends Serializable {
    public String getName ();
    public String getEmail ();
}
public interface RRecord extends Remote {
    public String getName () throws RemoteException;
    public String getEmail () throws RemoteException;
}
```

The registering of a person (add() method) relies on serialization to send to the server a copy of an object which implements the SRecord interface.

A lookup for a person (consult() method) returns a remote reference to a RMI object which implements the RRecord interface ; the applet can then use this remote reference to access the email address of the person with a remote invocation on that reference.

The consult() method includes a boolean parameter (*forward*) which indicates whether the request should be propagated to the other server if the person is not found (in order to prevent looping if the person does not exist).

You are given:

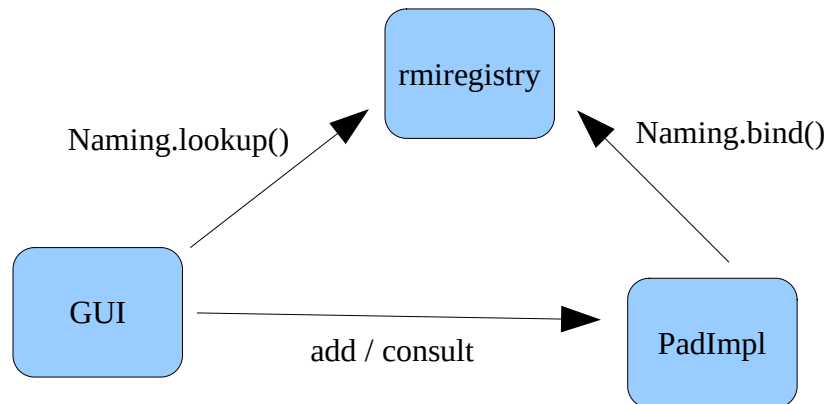
- Pad.java : the interface of the server
- SRecord.java : the interface of the object which is serialized and passed to the add() method
- RRecord.java : the interface of the RMI object returned by the consult() method
- GUI.java, java.policy, page.html : the applet which implements the GUI of the application

To run the applet:

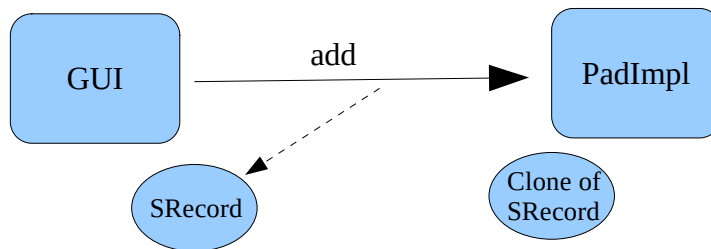
- in standalone mode: java GUI
- with the appletviewer : appletviewer -J-Djava.security.policy=java.policy page.html

You have to:

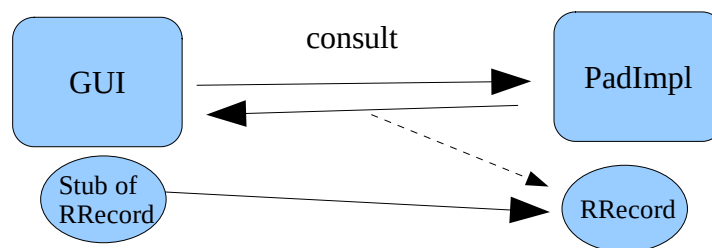
- implement PadImpl.java : the server class
- implement SRecordImpl.java : the class of the serialized object
- implement RRecordImpl.java : the class of the RMI object returned by the server
- modify GUI.java : to implement calls to the servers



When instantiated, the **PadImpl** registers (`bind()`) a stub in the **rmiregistry**. The **GUI** looks up the stub from the **rmiregistry**. It can then invoke methods (`add/consult`) on the **PadImpl**.



When `add()` is invoked, it takes as parameter a **SRecord** object which is serializable, so a copy of the object is passed.



When `consult()` is invoked, a **RRecord** object is returned, and since it is a remote object, a stub is returned, which makes the object accessible remotely from the **GUI**. The **GUI** can then invoke `getEmail()` remotely on the **RRecord** object.

```

1  import java.awt.*;
2  import java.awt.event.*;
3  import java.net.MalformedURLException;
4  import java.nio.charset.MalformedInputException;
5  import java.rmi.*;
6  import java.rmi.server.RemoteObject;
7
8  import javax.swing.*;
9
10 public class GUI extends JFrame {
11     TextField name, email;
12     Choice pads;
13     Label message;
14
15     public GUI() {
16         setSize(300, 200);
17         setLayout(new GridLayout(6, 2));
18         add(new Label(" Name : "));
19         name = new TextField(30);
20         add(name);
21         add(new Label(" Email : "));
22         email = new TextField(30);
23         add(email);
24         add(new Label(" Pad : "));
25         pads = new Choice();
26         pads.addItem("Pad1");
27         pads.addItem("Pad2");
28         add(pads);
29         add(new Label(""));
30         add(new Label(""));
31         Button Abutton = new Button("add");
32         Abutton.addActionListener(new AButtonAction());
33         add(Abutton);
34         Button Cbutton = new Button("consult");
35         Cbutton.addActionListener(new CButtonAction());
36         add(Cbutton);
37         message = new Label();
38         add(message);
39     }
40
41     class CButtonAction implements ActionListener {
42         public void actionPerformed(ActionEvent ae) {
43             String n, c;
44             n = name.getText();
45             c = pads.getSelectedItem();
46             message.setText("consult(" + n + "," + c + ")");
47
48             try {
49                 Pad pad = (Pad)Naming.lookup("//localhost:8080/"+c);
50                 RRecord record = pad.consult(n, true);
51                 message.setText("Nom : "
52                     + record.getName()
53                     + " | Email : "

```

```

54         + record.getEmail());
55     } catch (NotBoundException | MalformedURLException exc) {
56         exc.printStackTrace();
57     } catch (RemoteException exc) {
58         message.setText("Nom non trouvé !");
59     }
60 }
61 }
62
63 class AButtonAction implements ActionListener {
64     public void actionPerformed(ActionEvent ae) {
65         String n, e, c;
66         n = name.getText();
67         e = email.getText();
68         c = pads.getSelectedItem();
69         message.setText("add(" + n + ", " + e + ", " + c + ")");
70
71         try {
72             Pad pad = (Pad)Naming.lookup("//localhost:8080/"+c);
73             pad.add(new SRecordImpl(n, e));
74         } catch (RemoteException | NotBoundException | MalformedURLException exc) {
75             exc.printStackTrace();
76         }
77     }
78 }
79
80 public static void main(String args[]) {
81     GUI s = new GUI();
82     s.setSize(400, 200);
83     s.setVisible(true);
84 }
85 }
86
87 import java.rmi.*;
88
89 public interface Pad extends Remote {
90     public void add(SRecord sr) throws RemoteException;
91
92     public RRecord consult(String n, boolean forward) throws RemoteException;
93 }
94
95 import java.rmi.RemoteException;
96 import java.rmi.server.UnicastRemoteObject;
97 import java.util.HashMap;
98
99 public class PadImpl extends UnicastRemoteObject implements Pad {
100
101     private HashMap<String, String> pad;
102
103     public PadImpl() throws RemoteException {
104         this.pad = new HashMap<String, String>();
105     }
106
107     public void add(SRecord sr) throws RemoteException {

```

```

108         this.pad.put(sr.getName(), sr.getEmail());
109     }
110
111     public RRecord consult(String n, boolean forward) throws RemoteException {
112         if (this.pad.containsKey(n)) {
113             return new RRecordImpl(n, this.pad.get(n));
114         } else {
115             throw new RemoteException("L'email n'existe pas.");
116         }
117     }
118 }
119
120 import java.rmi.*;
121
122 public interface RRecord extends Remote {
123     public String getName() throws RemoteException;
124     public String getEmail() throws RemoteException;
125 }
126
127 import java.io.Serializable;
128 import java.rmi.RemoteException;
129 import java.rmi.server.UnicastRemoteObject;
130
131 public class RRecordImpl extends UnicastRemoteObject implements RRecord {
132
133     private String name;
134     private String email;
135
136     public RRecordImpl(String name, String email) throws RemoteException {
137         this.name = name;
138         this.email = email;
139     }
140
141     public String getName() throws RemoteException {
142         return name;
143     }
144
145     public String getEmail() throws RemoteException {
146         return email;
147     }
148 }
149
150 import java.rmi.Naming;
151 import java.rmi.registry.LocateRegistry;
152
153 public final class Server {
154
155     public static final String serverURI = "://localhost:8080";
156
157     public static void main(String args[]) {
158         try {
159             LocateRegistry.createRegistry(8080);
160             PadImpl pad1 = new PadImpl();
161             PadImpl pad2 = new PadImpl();

```

```
162         Naming.bind(serverURI+"/Pad1", pad1);
163         Naming.bind(serverURI+"/Pad2", pad2);
164     } catch (Exception e) {
165         e.printStackTrace();
166     }
167 }
168 }
169
170 import java.io.*;
171
172 public interface SRecord extends Serializable {
173     public String getName();
174     public String getEmail();
175 }
176
177 public class SRecordImpl implements SRecord {
178     private String name;
179     private String email;
180
181     public SRecordImpl(String name, String email) {
182         this.name = name;
183         this.email = email;
184     }
185
186     public String getName() {
187         return name;
188     }
189
190     public String getEmail() {
191         return email;
192     }
193 }
```